Astro SDK và tính nhất quán của dữ liệu thiên văn đa ngôn ngữ

7MLabs · Paper nghiên cứu kỹ thuật · Phiên bản 1.0 · 22 tháng 10 năm 2026

Tóm tắt

Astro SDK tổ chức phép tính thiên văn và các cấu trúc dữ liệu phục vụ ứng dụng chiêm tinh trên một lõi Rust dùng chung. Nghiên cứu này xem xét các hợp đồng tại ranh giới thời gian, thư viện thiên văn, giao diện ngôn ngữ và bước chuẩn bị dữ liệu cho mô hình ngôn ngữ lớn. Phương pháp gồm phân tích mã nguồn phiên bản 1.0.0, chạy bộ kiểm thử Rust hiện có, đối chiếu kinh độ với dữ liệu tham chiếu lưu trong dự án và đo nén trên toàn bộ 17 payload mẫu. Trong môi trường macOS ARM64 được ghi nhận, 132 kiểm thử đạt; cả 17 mẫu được khôi phục đúng giá trị JSON sau chế độ compact. Tổng kích thước giảm từ 20.014.085 xuống 11.554.591 byte, tương đương 42,27%; tuy nhiên hai truy vấn nhỏ tăng lần lượt 18,76% và 43,11%. Sai lệch kinh độ lớn nhất so với fixture Skyfield là 0,0001213 độ trên bốn trường hợp chứa ba thời điểm khác nhau. Kết quả hỗ trợ việc sử dụng các hợp đồng dữ liệu tường minh và đo theo từng payload, chưa chứng minh hiệu năng đa nền tảng, mức tiết kiệm token hoặc giá trị dự báo của chiêm tinh.

Từ khóa: Astro SDK; Rust; Swiss Ephemeris; hợp đồng dữ liệu; thời gian dân sự; nén JSON.

1 Giới thiệu

Một SDK dùng trong nhiều ngôn ngữ phải giữ được ý nghĩa của cùng một đầu vào khi đi qua các bộ tuần tự hóa, cơ chế quản lý bộ nhớ và thư viện native khác nhau. Ephemeris là mô hình hoặc dữ liệu vị trí thiên thể theo thời gian. Trong tính toán thiên văn, thời điểm, hệ quy chiếu và mô hình ephemeris còn quyết định cách diễn giải một tọa độ. Sai lệch có thể bắt đầu trước phép tính, chẳng hạn khi một giờ địa phương xuất hiện hai lần do đổi giờ mùa hè (DST), hoặc xuất hiện sau phép tính khi một bước rút gọn bỏ mất dữ kiện được tham chiếu.

Astro SDK của 7MLabs triển khai một lõi Rust, provider Swiss Ephemeris và các giao diện Node.js, Python, .NET, Rust và C. Các mô-đun tạo lá số, quan hệ, sự kiện và báo cáo lĩnh vực trả dữ liệu có cấu trúc; lời luận giải thuộc ứng dụng sử dụng SDK. Trong bài này, “fact” là dữ kiện do hệ thống tính hoặc tổ chức theo quy tắc đã khai báo, không mặc nhiên là một kết luận đã được kiểm chứng về tính cách hay cuộc sống. “Evidence” trong payload là quan hệ truy nguyên giữa các dữ kiện, khác với bằng chứng thực nghiệm cho hiệu lực của chiêm tinh. [1]

Nghiên cứu đặt ba câu hỏi. RQ1: các ranh giới provider, thời gian và giao diện native kiểm soát những dạng sai lệch nào? RQ2: biểu diễn compact bảo toàn giá trị JSON và thay đổi kích thước ra sao trên corpus mẫu của dự án? RQ3: bộ kiểm thử và fixture tham chiếu hiện có cho phép kết luận gì về triển khai đang xét? Câu hỏi được giới hạn ở tính chất phần mềm và dữ liệu, không đánh giá lời luận giải, tương hợp cá nhân hoặc khả năng dự báo sự kiện đời sống.

Đóng góp của bài là phân tích các hợp đồng có thể truy về mã nguồn; một phép đo nén có giữ lại kết quả bất lợi; và sự phân tách giữa kiểm tra hồi quy, tham chiếu thiên văn lưu sẵn và bằng chứng độc lập còn thiếu. Đây là nghiên cứu trường hợp kỹ thuật về một triển khai cụ thể. Bài không tuyên bố thuật toán thiên văn mới hoặc tính ưu tiên so với toàn bộ các SDK khác.

2 Nền tảng kỹ thuật và công trình liên quan

Swiss Ephemeris cung cấp các hàm tính tọa độ, nhà và chuyển đổi thời gian. Tài liệu chính thức phân biệt lựa chọn ephemeris, các cờ hệ tọa độ và giá trị trả về; trường hợp thiếu dữ liệu có thể dẫn tới lựa chọn mô hình thay thế nếu ứng dụng không kiểm tra cờ kết quả. Astro sử dụng thư viện này thay vì xây lại mô hình chuyển động thiên thể. Phần trách nhiệm bổ sung của SDK nằm ở kiểm tra đầu vào, khóa trạng thái, giới hạn phương án tính và công bố metadata. [2]

Skyfield là thư viện Python tính vị trí thiên thể, có thể dùng ephemeris JPL. Tài liệu của Skyfield phân biệt vị trí astrometric và apparent, cho thấy việc so sánh hai kinh độ cần thống nhất quy ước trước khi diễn giải sai số. Fixture của Astro ghi Skyfield 1.49, JPL DE421, cùng thời gian địa cầu (TT) và tọa độ apparent geocentric ecliptic-of-date. Phép đối chiếu trong bài sử dụng các giá trị đã lưu này; không lấy tuyên bố độ chính xác trên trang giới thiệu Skyfield làm độ chính xác của Astro. [3, 4]

Node-API cung cấp giao diện nhị phân ứng dụng (ABI) ổn định trong phạm vi API của Node.js, nhưng tài liệu nêu rõ bảo đảm đó không tự bao phủ các thư viện ngoài mà addon sử dụng. Vì vậy một lõi dùng chung không loại bỏ yêu cầu kiểm tra binary theo hệ điều hành và kiến trúc CPU. Bài xem Node-API là cơ chế tích hợp, không phải bằng chứng rằng mọi bản build sẽ trả cùng kết quả. [5]

IANA Time Zone Database lưu lịch sử độ lệch UTC và quy tắc giờ địa phương. Cơ sở dữ liệu được cập nhật khi quy định thay đổi; việc đóng gói dữ liệu cố định giúp tái lập nhưng đòi hỏi cập nhật có chủ đích. Astro dùng cơ sở dữ liệu đi kèm thay vì tra cấu hình múi giờ của máy đang chạy. Đây là một lựa chọn giữa tái lập theo phiên bản và phản ánh thay đổi mới nhất, không phải một bảo đảm về quy tắc múi giờ trong tương lai. [6, 7]

Các nguồn ngoài được chọn bằng cách mở tài liệu chính thức liên quan trực tiếp tới hợp đồng cần phân tích. Đây là tổng hợp kỹ thuật có chọn lọc, không phải tổng quan hệ thống. Chưa có thử nghiệm so sánh Astro với một SDK đối thủ, chưa đo lợi ích bảo trì của Rust so với các lõi triển khai riêng và chưa khảo sát đầy đủ văn liệu về nén ngữ cảnh cho mô hình ngôn ngữ lớn.

3 Phương pháp nghiên cứu

3.1 Đối tượng và phạm vi

Đối tượng là checkout sạch tại commit cd768d0192cd7e6f7aff266b8ebdb3f8685eeb39, có workspace version 1.0.0. Mã nguồn sản phẩm không được thay đổi trong quá trình nghiên cứu. Phân tích tập trung vào astro-provider-swiss, astro-time, astro-ffi, phần nén của astro-core và dữ liệu tham chiếu. Các mô-đun khác được kiểm tra thông qua bộ test hiện có, không được tuyên bố đã đánh giá chuyên sâu từng thuật toán. [1, 7, 8]

Trước phép đo, đề cương nội bộ xác định toàn bộ tệp examples/*-result.json làm corpus, công thức giảm byte và quy tắc giữ các kết quả bất lợi. Đề cương được lập sau lượt đọc README và tài liệu kiến trúc; không phải đăng ký trước nghiên cứu. Corpus gồm 17 tệp có engineVersion nguồn là 0.10.0-alpha.1. Do đó phép đo đánh giá bộ nén hiện tại trên payload lịch sử, không phải 17 payload được sinh mới bởi engine 1.0.0. [9]

3.2 Kiểm tra thực thi và tham chiếu

Bộ kiểm thử được chạy bằng cargo test --workspace --locked trên workspace neutral-engine. Chương trình dòng lệnh (CLI) Rust trong examples/rust được build bằng lockfile của chính ví dụ. Môi trường ghi nhận macOS 26.1 ARM64, rustc 1.93.1 và cargo 1.93.1; bản build CLI dùng profile dev. Nghiên cứu không sử dụng thời gian hoàn tất test làm phép đo hiệu năng. Log đầy đủ được lưu cùng mã phân tích. [9]

Với fixture natal, CLI tính lại bốn request được lưu trong tests/conformance/natal-references.json. Mỗi request có mười thiên thể. Kinh độ được đối chiếu với hai bộ giá trị lưu sẵn: pyswisseph 2.10.3.2 dùng Swiss 2.10.03/Moshier và Skyfield 1.49 dùng DE421. Sai lệch góc tròn được tính bằng giá trị nhỏ hơn giữa d và 360 − d, với d là phần dư của |a − b| theo 360, đơn vị độ. Chỉ kinh độ được đo lại trong phân tích này. [4, 9]

Bốn request chứa ba thời điểm khác nhau vì trường hợp năm 2000 được dùng cho cả Placidus và Whole Sign. Hai hàng này không tạo thành hai quan sát thiên văn độc lập. Các ngưỡng sẵn có trong fixture là 0,00001 độ cho Swiss và 0,01 độ cho tham chiếu độc lập; bài báo cả sai lệch quan sát thay vì chỉ báo trạng thái vượt hay không vượt ngưỡng. Việc tạo lại dữ liệu bằng Skyfield và xác minh file DE421 từ đầu nằm ngoài lượt thực thi này. [4]

3.3 Phép đo nén và tiêu chí bảo toàn

Mỗi payload được đưa vào compress_json với mode compact, sau đó context được đưa vào expand_context_json. Giá trị JSON nguồn và giá trị giải nén được so sánh bằng bộ đọc số thập phân Decimal để tránh thêm dung sai số thực vào bước kiểm tra. Thứ tự khóa object, khoảng trắng và cách viết tương đương của số không phải tiêu chí bảo toàn; thứ tự phần tử mảng, giá trị số, chuỗi, null và cấu trúc dữ liệu vẫn được kiểm tra. Dữ liệu đi qua bộ đọc/ghi JSON của Python trước khi gọi CLI; kết luận không mở rộng sang JSON số nguyên tùy ý ngoài corpus này. [9]

Baseline là số byte UTF-8 của envelope nguồn sau khi core tuần tự hóa gọn. Kết quả là số byte của toàn bộ context native, gồm dictionary, coverage, omissions, metrics và warnings. Đếm byte đầu

ra được kiểm tra lại từ stdout của CLI sau khi bỏ ký tự xuống dòng do println thêm. Mức giảm R bằng 100 × (Bnguồn − Bcontext) / Bnguồn; R âm có nghĩa context lớn hơn nguồn. [8, 9]

Mỗi fixture được chạy một chu trình nén/giải nén. Phép đo là mô tả kích thước trên corpus cố định, không ước lượng phân phối tải production. Tổng giảm byte được tính từ tổng tử số và mẫu số, không lấy trung bình không trọng số của các phần trăm. Bài không so compact với gzip, không đo tokenizer, độ trễ, bộ nhớ hoặc chất lượng câu trả lời của mô hình ngôn ngữ.

4 Phân tích kiến trúc và hợp đồng

4.1 Một lõi tính toán và các ranh giới dữ liệu

Luồng chính đi từ đầu vào JSON qua kiểm tra yêu cầu, tính thiên văn, tổ chức dữ kiện và envelope đầu ra. Node-API gọi lõi native; Python và .NET đi qua C ABI. Cách tổ chức này cho phép các binding sử dụng cùng triển khai phép tính, nhưng lỗi ở wrapper, marshaling và đóng gói vẫn có thể xảy ra. Tính dùng chung của mã là quan sát kiến trúc; tính giống nhau của đầu ra giữa mọi binding là một thuộc tính phải đo riêng. [1, 7]

Trong C ABI, output được cấp phát dưới dạng CString của Rust và phải giải phóng đúng một lần bằng astro_free_string. Hàm invoke_json bắt panic Rust, xử lý con trỏ null và UTF-8 không hợp lệ bằng envelope lỗi. Một con trỏ native không hợp lệ vẫn vi phạm điều kiện an toàn của caller; catch_unwind không biến C ABI thành một giao diện an toàn với mọi địa chỉ bộ nhớ. Đây là giới hạn cụ thể của hợp đồng thay vì một bảo đảm an toàn tuyệt đối. [7]

4.2 Trạng thái provider và lựa chọn mô hình

Provider khai báo Mutex dùng chung trong từng native library. Lời gọi natal giữ khóa qua khởi tạo, chuyển đổi thời gian, tính vị trí, nhà và dọn trạng thái. EphemerisSession giữ khóa cho một phiên quét. Mã reset_provider đóng trạng thái trước, khởi tạo Delta T theo Moshier và đặt tham số tidal acceleration; phép tính vị trí kiểm tra kết quả hữu hạn và cờ mô hình trả về. Khóa làm rõ quyền truy cập trạng thái, đồng thời có thể giới hạn thông lượng trong cùng thư viện; tác động này chưa được đo. [7]

Khi phép tính nhà trả lỗi, mã từ chối trả kết quả thay vì công bố nhà dự phòng như thể đó là hệ được yêu cầu. Một vết lỗi có thể kiểm tra từ mã là: caller yêu cầu Placidus ở tọa độ không tính được; Swiss trả trạng thái lỗi; provider trả Err; kết quả thành công không được tạo từ các cusp dự phòng. Đây là phân tích nhánh xử lý, không phải một phép fault injection mới của nghiên cứu. [7]

Hồ sơ nguồn gốc pin Swiss Ephemeris 2.10.03 tại commit upstream aacf962d19d7; mã commit đầy đủ được lưu trong PROVENANCE.md [7]. Nhánh tính hành tinh cơ bản dùng Moshier và chặn đọc ephemeris hoặc bảng thời gian ngoài. Tuy nhiên “offline” không có nghĩa mọi điểm mở rộng đều không dùng dữ liệu ephemeris: dữ liệu seas_18.se1 phục vụ Chiron được nhúng trong binary theo provenance. Mô tả chính xác là không cần tải dữ liệu khi tính trong các nhánh được tài liệu hóa. [7]

4.3 Thời gian dân sự và phạm vi dữ kiện

astro-time yêu cầu đúng một nguồn định vị thời gian: tên IANA hoặc offset nguyên theo giây. Giờ không tồn tại trong một lần chuyển DST bị từ chối; giờ xuất hiện hai lần mặc định bị từ chối cho tới khi caller chọn earlier hoặc later. Mô-đun còn cảnh báo về dữ liệu lịch sử trước năm 1970 và quy tắc tương lai. Việc trả UTC sau khi lựa chọn không xác nhận rằng lựa chọn earlier/later đúng với hồ sơ sinh; trách nhiệm đó vẫn ở nguồn dữ liệu đầu vào. [7]

Profile lĩnh vực chọn và tổ chức các dữ kiện từ biểu đồ, còn bộ nén thay cách biểu diễn hoặc phạm vi dữ liệu được chuyển tiếp. Hai thao tác này không thay thế thẩm định ý nghĩa của quy tắc chiêm tinh. Việc tách lớp cho phép kiểm tra tọa độ, lựa chọn dữ kiện và câu diễn giải bằng các tiêu chí khác nhau, thay vì gán một trạng thái “đúng” chung cho toàn bộ chuỗi. [1, 8]

4.4 Biểu diễn compact và lựa chọn phạm vi

Bộ nén dùng bảng cho các object có cấu trúc lặp, dictionary cho giá trị lặp và marker literal để bảo toàn khóa nguồn trùng tên marker. compact giữ toàn bộ envelope ở mức giá trị JSON. focused và budgeted có thể bỏ domain hoặc section, đồng thời công bố phần bỏ trong omitted; chúng không có cùng cam kết khôi phục toàn bộ nguồn như compact. [8]

Mã lựa chọn giữ các cấu trúc dữ kiện chia sẻ thay vì cắt lại toàn bộ các chỉ mục tham chiếu. Đây là đánh đổi giữa giảm kích thước và giữ quan hệ dữ liệu. Nếu phạm vi đã chọn vẫn vượt ngân sách, budgeted báo vượt thay vì tự xóa facts cho vừa giới hạn. estimatedTokens hiện được gán từ số byte UTF-8; vì vậy không được diễn giải cột này như kết quả tokenizer hoặc ngân sách của toàn bộ prompt. [8]

5 Kết quả thực thi

5.1 Bộ kiểm thử hiện có

Lượt chạy cargo hoàn tất với 132 test đạt, không có test thất bại hoặc bị bỏ qua trong các nhóm được chạy. Tổng này được cộng từ các dòng kết quả trong log, không cộng các nhóm doc-test có số ca bằng không. Bộ test có kiểm tra nén/giải nén, đầu vào không hợp lệ, khởi tạo provider lạnh, thời gian mơ hồ và các hợp đồng composite, couple, forecast. Số ca không phải độ bao phủ mã hoặc bằng chứng vét cạn đầu vào. [9]

Tài liệu phát hành của dự án báo cáo 339 trường hợp đối chiếu binding cho bản 1.0.0. Nghiên cứu không chạy lại toàn bộ ma trận Node/Python/.NET/Rust/C hoặc các consumer Linux; con số đó chỉ là bằng chứng lịch sử do dự án ghi nhận. Kết quả mới trong bài là 132 test Rust và các phép đo CLI được mô tả ở mục 3. [10]

5.2 Bảo toàn dữ liệu và kích thước

Cả 17 mẫu đạt so sánh giá trị JSON sau giải nén; cả 17 số outputBytes khớp phép đếm stdout native. Tổng nguồn 20.014.085 byte được biểu diễn thành 11.554.591 byte, giảm 8.459.494 byte, tương đương 42,27%. Đây là kết quả có trọng số theo kích thước của corpus, chịu ảnh hưởng lớn bởi các báo cáo dài. [9]

Bảng 1. Kết quả compact trên toàn bộ corpus. Tên bỏ hậu tố -result.json; số byte tính trên JSON native gọn. R âm nghĩa kích thước tăng. Nguồn: phép chạy mới và results.json [9].

Mẫu	Nguồn byte	Context byte	R %
composite	2.206.657	1.374.846	37,70
couple-composite	4.126.966	2.353.132	42,98
couple	2.101.342	1.101.529	47,58
custom-composite	319.866	191.531	40,12
custom-couple	709.819	386.109	45,60
custom-forecast	411.884	201.689	51,03
custom-profiles	256.206	143.847	43,85
events-year	1.264.416	1.054.977	16,56
forecast-day	306.138	158.573	48,20
forecast-month	4.296.721	2.195.074	48,91
forecast-year	1.828.614	1.062.147	41,92
natal-domains	2.129.952	1.291.076	39,38
natal	7.147	5.269	26,28
query-aspects	1.908	2.266	-18,76
query-between	900	1.288	-43,11
query-houses	28.666	18.636	34,99
query-points	16.883	12.602	25,36

Hai truy vấn query-aspects và query-between tăng lần lượt 358 và 388 byte. Chi phí biểu diễn và metadata có thể lớn hơn phần lặp được loại bỏ trong payload ngắn. Các kết quả này bác bỏ phát biểu

“compact luôn làm nhỏ dữ liệu” ngay trong corpus nghiên cứu. Ngược lại, custom-forecast giảm 51,03%, cho thấy mức giảm phụ thuộc cấu trúc cụ thể chứ không chỉ tên operation. [9]

Các số byte đầu ra trong bài khác một số bảng lịch sử của dự án. Bộ nén hiện tại ghi phiên bản 1.0.0 ở envelope ngoài, trong khi payload nguồn giữ phiên bản cũ. Không dùng sai khác số byte giữa hai bảng như bằng chứng cải tiến thuật toán nếu chưa kiểm soát phiên bản metadata và điều kiện đo. [8, 9]

5.3 Đối chiếu kinh độ với fixture

Trên bốn request, kinh độ tính lại khớp giá trị Swiss lưu sẵn tại các số được so sánh. So với Skyfield lưu sẵn, sai lệch lớn nhất là 0,0001213184 độ, khoảng 0,437 giây cung. Giá trị này nằm dưới ngưỡng 0,01 độ của fixture; ngưỡng đó là tiêu chí hồi quy của dự án, không phải chuẩn độ chính xác do bài thiết lập. [4, 9]

Bảng 2. Sai lệch kinh độ lớn nhất trong từng request, đơn vị độ. Mỗi hàng so mười thiên thể; hai hàng năm 2000 cùng thời điểm. Nguồn: tính lại bằng CLI so với fixture đã lưu [4, 9].

Trường hợp	Swiss	Skyfield
1950-6-15 placidus	0	0,0001071962
2000-1-1 placidus	0	0,0000716519
2020-2-29 placidus	0	0,0001213184
2000-1-1 wholeSign	0	0,0000716519

Kết quả Swiss khớp không cung cấp kiểm chuẩn độc lập vì hai phía dùng cùng họ thư viện và mô hình. Kết quả Skyfield có khác biệt phương pháp ở nguồn tham chiếu, nhưng nghiên cứu này chỉ kiểm tra các số lưu trong repository. Chưa kiểm tra lại quá trình tạo fixture, chưa mở rộng đến mọi ngày trong khoảng 1800–2399 và chưa đánh giá sai lệch của các điểm mở rộng hoặc thời điểm sự kiện bằng một oracle mới.

6 Thảo luận và giới hạn

RQ1 được trả lời bằng các ranh giới tường minh: lựa chọn thời gian không bị sửa ngầm, cờ ephemeris được kiểm tra, trạng thái native được khóa và quyền sở hữu chuỗi được công bố. Những cơ chế này làm cho một số lỗi có đường xử lý quan sát được. Chúng không loại bỏ sai dữ liệu nguồn, lỗi ngoài phạm vi test hoặc vấn đề ở binary trên nền tảng khác. Đặc biệt, khóa toàn phiên quét là một đánh đổi cần đo dưới tải đồng thời trước khi dự báo khả năng phục vụ.

RQ2 cho thấy compact bảo toàn các mẫu đã thử nhưng không luôn giảm byte. Ứng dụng nên quyết định dùng nguồn hay context sau khi đo payload thực và xét giao diện tiêu thụ. Nếu phía nhận yêu cầu astro-context/1 thì đổi sang JSON gốc còn cần thay đổi hợp đồng phía nhận; không thể quyết định chỉ dựa trên số byte. focused và budgeted phù hợp khi phạm vi câu hỏi đã được xác định, nhưng cần đọc omissions trước khi diễn giải kết quả. [8, 9]

RQ3 chỉ hỗ trợ kết luận hồi quy hữu hạn. Các fixture có thể chia sẻ cấu trúc, được tạo bởi chính dự án và không phải mẫu ngẫu nhiên từ người dùng. Bộ test tích lũy theo quá trình phát triển nên có nguy cơ bỏ sót dạng lỗi chưa từng được nghĩ tới. Các quan sát nén không độc lập với thiết kế schema; tỷ lệ tổng không thể dùng như kỳ vọng tiết kiệm cho một ứng dụng khác. Một lần chạy trên macOS ARM64 không suy rộng thành bảo đảm Linux, Windows hoặc môi trường trình duyệt.

Một lõi dùng chung giảm số nơi triển khai cùng công thức về mặt cấu trúc, nhưng nghiên cứu chưa đo công sức phát triển, chi phí sửa lỗi hoặc mức lệch giữa các đội ngũ. Tương tự, chạy cục bộ có thể tránh gửi dữ liệu tới dịch vụ tính toán ở bước SDK, song ứng dụng vẫn có thể gửi payload tới mô hình ngôn ngữ hoặc hệ thống lưu trữ. Bài không tiến hành audit mạng hay đánh giá quyền riêng tư toàn ứng dụng.

Các bước nghiên cứu tiếp theo nên ưu tiên tạo lại tham chiếu bằng nguồn độc lập với quy ước tọa độ và thời gian cố định; kiểm tra parity trên binary thực của từng target; và xây workload đồng thời để đo ảnh hưởng của khóa provider. Nếu mục tiêu là tiết kiệm chi phí mô hình ngôn ngữ lớn (LLM), cần token hóa prompt cuối cùng, giữ nguyên nhiệm vụ và đánh giá độ chính xác của câu trả lời cùng tỷ lệ thiếu căn cứ. Đây là các hướng đánh giá chưa thực hiện, không phải kết quả của paper.

7 Kết luận

Trong phạm vi mã nguồn và môi trường được xét, Astro SDK cung cấp một chuỗi xử lý có các hợp đồng tường minh từ thời gian dân sự đến dữ liệu thiên văn và context. Bộ kiểm thử hiện có đạt trong lượt chạy mới; compact khôi phục đúng cả 17 payload đã chọn và giảm tổng byte của corpus, đồng thời làm tăng kích thước ở hai truy vấn nhỏ. Các đối chiếu kinh độ với fixture hỗ trợ tính ổn định của những trường hợp đã kiểm tra, chưa thay thế kiểm chuẩn độc lập trên miền rộng. Hàm ý kỹ thuật là đánh giá riêng tính toán, chuyển dữ liệu và diễn giải, đồng thời quyết định nén bằng phép đo thực tế thay vì một tỷ lệ giảm mặc định.

Tài liệu tham khảo

[1] 7MLabs. Astro SDK 1.0.0, mã nguồn và tài liệu kiến trúc. Commit cd768d0192cd7e6f7aff266b8ebdb3f8685eeb39; README.vi.md, docs/architecture.md. Repository: https://github.com/7mlabs/sdk-astro . Nguồn được đọc từ checkout local ngày 22/10/2026.

[2] Astrodienst AG. Swiss Ephemeris Programming Interface. Các mục Ephemeris flags, Error handling, Date and time conversion. https://www.astro.com/swisseph/swephprg.htm . Truy cập 22/10/2026. Tài liệu trực tuyến dùng để giải thích API; phiên bản provider pin được ghi tại [7].

[3] Skyfield. Skyfield documentation và Positions. https://rhodesmill.org/skyfield/ ; https://rhodesmill.org/skyfield/positions.html . Truy cập 22/10/2026. Đã đọc phần giới thiệu và phân biệt các loại vị trí; không dùng tuyên bố độ chính xác của nguồn làm kết quả Astro.

[4] 7MLabs. Natal reference fixtures. tests/conformance/natal-references.json, tại commit [1]. Metadata nguồn: pyswisseph 2.10.3.2, Swiss 2.10.03, Skyfield 1.49, JPL DE421. Dữ liệu tham chiếu lưu sẵn, đọc 22/10/2026.

[5] Node.js. Node-API, mục Implications of ABI stability. https://nodejs.org/api/n-api.html . Truy cập 22/10/2026. Nguồn mô tả phạm vi ổn định ABI, không thay kết quả kiểm thử binary của Astro.

[6] IANA. Time Zone Database. https://www.iana.org/time-zones . Truy cập 22/10/2026. Nguồn mô tả lịch sử giờ địa phương và việc cập nhật quy tắc.

[7] 7MLabs. Provider, time và native interface implementation. neutral-engine/crates/astro-provider-swiss/src/lib.rs, PROVENANCE.md, tests/initialization.rs; astro-time/src/lib.rs; astro-ffi/src/lib.rs; cùng commit [1]. Đọc 22/10/2026.

[8] 7MLabs. Payload compression contract và implementation. docs/payload-compression.md; neutral-engine/crates/astro-core/src/compression.rs; cùng commit [1]. Đọc 22/10/2026.

[9] Hồ sơ nghiên cứu đi kèm paper, 22/10/2026. evidence/analyze.py, results.json, cargo-test.log, analysis.log và natal-0.json đến natal-3.json, trong docs/whitepapers/astro-sdk. Mã phân tích và kết quả local, chưa được công bố thành một bộ dữ liệu độc lập.

[10] 7MLabs. Kết quả cài đặt và kiểm thử trực tiếp, mục SDK 1.0.0. docs/testing.md tại commit [1]. Báo cáo phát hành của dự án, đọc 22/10/2026; ma trận binding chưa được chạy lại trong nghiên cứu này.

Dữ liệu và các thông tin công bố

Mã phân tích, hash SHA-256 của từng fixture và log thực thi được bàn giao cùng bản paper để người có checkout tương ứng kiểm tra lại. Nghiên cứu sử dụng dữ liệu mẫu đã có trong repository, không thu thập hồ sơ sinh của người tham gia mới. Repository được nêu để nhận diện dự án; bộ kết quả mới hiện chỉ lưu local, không được ngụ ý đã phát hành công khai. [9]

7MLabs là đơn vị biên soạn theo yêu cầu. Danh tính tác giả cá nhân chịu trách nhiệm, nguồn tài trợ và khai báo lợi ích liên quan chưa được xác nhận. AI hỗ trợ đọc mã, chạy công cụ, tổng hợp và biên soạn; chưa có phản biện chuyên môn độc lập. Tài liệu là bản thảo kỹ thuật có kết quả thực thi trong phạm vi nêu trên, chưa phải tuyên bố được chấp nhận ở một tạp chí hoặc kho preprint.